

Working Memory Layer — v2

Bug Fix Edition · Mapped to Actual Codebase State After Sprint 3

Gaurav Kumar · June 11, 2026 · Based on codebase audit report

The codebase audit revealed that Sprints 1–3 are partially implemented but have three critical bugs that are causing: (1) old chat bubbles vanishing from the UI, (2) the model not remembering things reliably, and (3) slow responses on long conversations. This document explains each bug, gives the exact fix, and provides the remaining sprint tasks updated to reflect what actually exists in your code right now.

AUDIT — What the Codebase Audit Found — Reality vs Plan

What is already correctly implemented

- ✓ rolling_summary accepted in ChatRequest (chat.py line 18) — field exists
- ✓ rolling_summary injected into prompt before LLM call (chat.py line 55)
- ✓ Token budget pre-flight wired into chat.py via build_payload_within_budget
- ✓ Async compression fork with asyncio.create_task (chat.py line 96)
- ✓ SSE control frame injection (chat.py line 121)
- ✓ memory_compact handler exists in ChatPage.jsx (line 998)
- ✓ createEmptySession has rolling_summary field (ChatPage.jsx line 37)
- ✓ backend/utis/memory_utils.py exists
- ✓ backend/services/compression_service.py exists

Three bugs found — causing all reported symptoms

- BUG 1 (Critical) — Display and payload use the SAME messages array. When memory_compact arrives, messages.slice(truncated_count) prunes both the API payload AND the visible chat bubbles. Old messages vanish from the UI.
- BUG 2 (Critical) — Naming mismatch: session state uses rolling_summary but POST body reads activeSession.rollingSummary (camelCase). Before first compression, rollingSummary is undefined, sending empty string. After compression both are set but the camelCase key is the one being read — fragile and inconsistent.
- BUG 3 (Serious) — chat.py exception handler has invalid code: isinstance(e, 'detail') — second argument to isinstance must be a type, not a string. This will raise a TypeError and crash the handler silently.

Why responses get slower on long conversations

- No ROLLING_LIMIT exists in ChatPage.jsx — there is no slice(-20) on the payload. The frontend sends ALL messages on every turn even though the plan said to cap at 20. Message 60 sends 60 messages worth of tokens. The LLM input grows linearly with conversation length, which is exactly why response time is increasing.

The Problem in Detail

Your React session object has one messages array. The UI renders chat bubbles from it at line 1267. The API payload is built from it at line 886. When memory_compact arrives at line 998 and does messages.slice(truncated_count), it removes old messages from the same array the UI uses. The user sees their chat history disappear.

The Fix — two arrays, one session

Split into displayMessages (full history, UI only, never pruned) and contextMessages (rolling window, API payload only, pruned on compression). The user always sees everything. The LLM only gets the efficient window.

Step 1 — Update createEmptySession

File: frontend/src/ChatPage.jsx · createEmptySession (~line 37)

```
// REPLACE createEmptySession with: function createEmptySession() { const sessionId = createSessionId(); return
{ id: sessionId, session_id: sessionId, title: 'New Chat', startedAt: new Date(), displayMessages: [], // <--
NEW: full history, rendered in UI, never pruned contextMessages: [], // <-- NEW: rolling window, sent to LLM,
pruned on compact rolling_summary: "", // keep snake_case only – remove rollingSummary compression_epoch: 0,
summary_history: [], hasUserTitle: false, pinned: false, archived: false, }; }
```

Step 2 — Update all places that read/write messages

File: frontend/src/ChatPage.jsx · ~line 527 (where messages is derived)

```
// BEFORE: const messages = activeSession?.messages || []; // AFTER – two separate derived arrays: const
displayMessages = activeSession?.displayMessages || []; // for UI rendering const contextMessages =
activeSession?.contextMessages || []; // for API payload
```

Step 3 — Update sendMessage to append to BOTH arrays

File: frontend/src/ChatPage.jsx · ~line 886 (sendMessage / nextPageMessages)

```
// BEFORE: const nextMessages = [...messages, userMessage]; // AFTER: const nextDisplayMessages =
[...displayMessages, userMessage]; const nextContextMessages = [...contextMessages, userMessage]; // When
adding assistant reply after stream, also update both: // displayMessages gets every message (for UI) //
contextMessages gets every message until compaction prunes it
```

Step 4 — Update UI rendering to use displayMessages

File: frontend/src/ChatPage.jsx · ~line 1267 (chat bubble render)

```
// BEFORE: {messages.map((message) => { // AFTER: {displayMessages.map((message) => {
```

Step 5 — Fix the memory_compact handler — prune contextMessages only

File: frontend/src/ChatPage.jsx · ~line 998 (memory_compact handler)

```
// BEFORE (this is the bug – prunes display messages): if (parsed.control === "memory_compact") {
setSessions((currentSessions) => currentSessions.map((session) => session.id === activeSession.id ? {
...session, rolling_summary: parsed.rolling_summary, rollingSummary: parsed.rolling_summary, // remove this
messages: session.messages.slice(parsed.truncated_count), // BUG } : session, ), ); continue; } // AFTER – only
prune contextMessages, displayMessages stays untouched: if (parsed.control === "memory_compact") {
setSessions((currentSessions) => currentSessions.map((session) => { if (session.id !== activeSession.id) return
session; const newHistory = [ ...( session.summary_history || [] ).slice(-2), session.rolling_summary,
].filter(Boolean); return { ...session, rolling_summary: parsed.rolling_summary, // snake_case only
contextMessages: session.contextMessages.slice(parsed.truncated_count), // displayMessages: UNTOUCHED – user
sees full history always compression_epoch: (session.compression_epoch || 0) + 1, summary_history: newHistory,
}; } ) ); continue; }
```

Step 6 — Fix the POST payload to send contextMessages

File: frontend/src/ChatPage.jsx · ~line 919 (fetch POST body)

```
// BEFORE: const nextMessages = [...messages, userMessage]; // ... body: JSON.stringify({ ... rolling_summary:
activeSession.rollingSummary || "", // BUG: camelCase messages: nextMessages.map(({ role, content:
messageContent }) => ({ role, content: messageContent, })), }) // AFTER: const ROLLING_LIMIT = 20; const
nextContextMessages = [...contextMessages, userMessage]; const payloadMessages =
nextContextMessages.slice(-ROLLING_LIMIT); body: JSON.stringify({ provider: provider.provider, api_key:
provider.apiKey, model_name: provider.modelName, rolling_summary: activeSession.rolling_summary || "", //
```

```
snake_case, consistent compression_epoch: activeSession.compression_epoch || 0, summary_history:
activeSession.summary_history || [], messages: payloadMessages.map(({ role, content: messageContent }) => ({
role, content: messageContent, })), })
```

■ After this fix: user always sees full chat history. LLM receives only the rolling window. Compression silently prunes contextMessages only. Zero UI flicker.

The Problem

Your session state sets `rolling_summary` (snake_case) in `createEmptySession`. But the POST body reads `activeSession.rollingSummary` (camelCase). JavaScript returns undefined for a key that doesn't exist — so the POST body sends empty string for `rolling_summary` on every turn until the first compression fires and sets `rollingSummary` explicitly. Even after that, having both keys is a time bomb for future bugs.

Evidence from audit

```
// createEmptySession sets: rolling_summary: "" // snake_case ✓ // POST body reads: rolling_summary:
activeSession.rollingSummary || "" // camelCase ✗ – undefined before first compact // memory_compact handler
sets BOTH (fragile): rolling_summary: parsed.rolling_summary, // snake_case rollingSummary:
parsed.rolling_summary, // camelCase – should not exist
```

The Fix — use snake_case everywhere, delete camelCase

Fix 2 is already done in Fix 1 above (Step 5 and Step 6). To be explicit, here is the complete checklist:

- `createEmptySession`: keep `rolling_summary`, delete `rollingSummary` — done in Fix 1 Step 1
- `memory_compact` handler: only set `rolling_summary`, remove `rollingSummary` line — done in Fix 1 Step 5
- POST body: read `activeSession.rolling_summary` (not `.rollingSummary`) — done in Fix 1 Step 6
- Search entire `ChatPage.jsx` for `rollingSummary` — delete every occurrence

Search and replace command to run in your editor

```
# In your editor find-and-replace (case sensitive): # Find: rollingSummary # Replace: rolling_summary # File:
frontend/src/ChatPage.jsx only # Then audit: grep -n "rollingSummary" frontend/src/ChatPage.jsx # Result should
be: no matches
```

FIX 3 — isinstance TypeError in chat.py Exception Handler

Silent crash bug

The Problem

The exception handler in `chat.py` contains: `isinstance(e, 'detail')`. The second argument to `isinstance` must be a type or tuple of types — not a string. Python raises `TypeError: isinstance() arg 2 must be a type, a tuple of types, or a union`. This means any exception in the stream handler that hits this branch will produce a secondary error, masking the original error and making debugging very hard.

File: `backend/routers/chat.py` · ~line 135

```
# FIND this broken code (line ~135): except Exception as e: if isinstance(e, "detail"): # BUG – "detail" is a
string, not a type ... # REPLACE with correct exception handling: except Exception as e: error_msg = str(e) #
Log the real error for debugging print(f"[chat] Stream error: {type(e).__name__}: {error_msg}") # Send error to
frontend as SSE event import json yield f"data: {json.dumps({'error': error_msg})}\n\n"
```

Why responses get slower

The audit confirmed: there is no `slice(-20)` or `ROLLING_LIMIT` anywhere in `ChatPage.jsx`. The POST payload sends ALL messages on every turn — message 40 sends 40 messages, message 60 sends 60. LLM time-to-first-token scales directly with input token count. This is why your early messages are fast and later messages are slow.

This fix is already included in Fix 1 Step 6. Specifically:

```
// In the POST payload block (Fix 1 Step 6): const ROLLING_LIMIT = 20; const payloadMessages =
nextContextMessages.slice(-ROLLING_LIMIT); // This caps the payload at 20 messages regardless of conversation
length. // Combined with rolling_summary, the LLM always has full context but // never receives more than 20 raw
messages + the summary.
```

Expected result after fix

- Message 5 response time: same as before
- Message 40 response time: same as message 5 — payload is always capped
- Message 100 response time: same as message 5 — compression keeps context lean

FIX 5 — memory_working.py — Broken verify-provider Endpoint

Prevents production 500 errors

The Problem

`memory_working.py` line 192 references `PROVIDER_MODULES` and `get_provider_module`. The audit confirmed neither is defined anywhere in the codebase. The `/api/verify-provider` endpoint will raise `NameError` if called. The route exists in `memory_working.py` — it is registered through `routers/_init_.py`. `auth.py` may also have this route, causing unpredictable routing.

File: `backend/routers/memory_working.py` · ~line 183 (verify_provider endpoint)

```
# FIND the broken verify_provider function: @router.post("/verify-provider") async def verify_provider(payload:
VerifyRequest): try: resolved_config = await resolve_provider_config(payload) provider_name =
resolved_config["provider"] api_key = resolved_config["api_key"] model_name = resolved_config["model_name"] if
provider_name not in PROVIDER_MODULES: # NameError here raise ValueError(f"Unsupported provider:
{provider_name}") provider_module = get_provider_module(provider_name) # NameError here success, message =
provider_module.verify(api_key, model_name) return {"success": success, "message": message} except Exception as
e: raise HTTPException(status_code=400, detail=str(e)) # REPLACE WITH – use ProviderFactory which is already
imported: @router.post("/verify-provider") async def verify_provider(payload: VerifyRequest): try:
resolved_config = await resolve_provider_config(payload) provider_name = resolved_config["provider"] api_key =
resolved_config["api_key"] model_name = resolved_config["model_name"] if not provider_name: raise
ValueError("No provider specified") # Use the existing ProviderFactory instead of undefined PROVIDER_MODULES
provider_key = f"{provider_name}:{api_key}" provider_instance = ProviderFactory.create(provider_key, api_key) #
Send a minimal test message to verify the connection test_messages = [{"role": "user", "content": "Hi"}] tokens
= [] async for chunk in provider_instance.generate_stream(model_name, test_messages): tokens.append(chunk) if
len(tokens) > 3: # don't need full response, just confirm it works break return {"success": True, "message":
f"Provider {provider_name} verified successfully"} except Exception as e: raise HTTPException(status_code=400,
detail=str(e))
```

Also check for duplicate route conflict with auth.py

```
# Run this to see if /api/verify-provider is defined in both files: # grep -rn "verify-provider" backend/ # If
it appears in both memory_working.py and auth.py: # Keep it in ONE place only – memory_working.py since that's
where # the memory config logic lives. # In auth.py, DELETE the verify-provider route if it exists there.
```

Updated session state shape (ChatPage.jsx)

```
// Final createEmptySession shape: { id: sessionId, session_id: sessionId, title: 'New Chat', startedAt: new
Date(), // DISPLAY — full history, rendered in UI, never pruned displayMessages: [], // CONTEXT — rolling
window, sent to LLM, pruned on memory_compact contextMessages: [], // MEMORY rolling_summary: "", // snake_case
only — no rollingSummary compression_epoch: 0, summary_history: [], // UI flags hasUserTitle: false, pinned:
false, archived: false, }
```

Data flow after all fixes

Event	displayMessages	contextMessages	rolling_summary
User sends message	Append userMessage	Append userMessage	No change
POST payload built	Not sent	slice(-20) sent as messages[]	Sent as rolling_summary
Assistant reply arrives	Append assistantMessage	Append assistantMessage	No change
memory_compact received	NOT TOUCHED — full history preserved (truncated_count) — pruned		Updated to new_summary
User refreshes page	Lost (no persistence yet)	Lost (no persistence yet)	Lost (no persistence yet)

What each symptom looks like after fixes

Symptom	Root Cause	Fixed By	Expected After Fix
Old chats vanishing	memory_compact pruned displayMessages	Messages split arrays	UI shows full history always
Model not remembering	rollingSummary undefined, sent as empty string	Fixed by snake_case every time	rolling_summary sent correctly every turn
Slow long responses	No ROLLING_LIMIT, payload grows infinitely	Fixed by slice(-20)	Consistent response time at any length
500 on verify-provider	PROVIDER_MODULES undefined	Fix 5 — use ProviderFactory	Clean 200 response
Masked stream errors	isinstance(e, 'detail') TypeError	Fix 3 — correct exception handling	Real errors logged and surfaced

REF — Master Fix Task Table — Ordered by Priority

Priority	Task	File	Line	Time
1 — Do first	Split displayMessages / contextMessages in createEmptySession	ChatPage.jsx	~37	15 min
1 — Do first	Update UI render to use displayMessages	ChatPage.jsx	~1267	2 min
1 — Do first	Update sendMessage to write to both arrays	ChatPage.jsx	~886	10 min
1 — Do first	Fix memory_compact handler — prune contextMessages only	ChatPage.jsx	~998	10 min
1 — Do first	Fix POST body — read rolling_summary, add ROLLING_LIMIT	ChatPage.jsx	~919	10 min
2 — Same session	Grep and remove all rollingSummary references	ChatPage.jsx	all	5 min
3 — Same session	Fix isinstance(e, 'detail') in exception handler	chat.py	~135	5 min
4 — Next session	Fix verify_provider to use ProviderFactory	memory_working.py	~183	15 min
4 — Next session	Check and remove duplicate /api/verify-provider in auth.py	auth.py	check	5 min
5 — Sprint 4	Add compression_epoch + summary_history to ChatRequest	chat.py	~18	5 min
5 — Sprint 4	Add grounding_pass function to compression_service.py	compression_service.py	add	30 min
5 — Sprint 4	Trigger grounding pass every 5th epoch in chat.py	chat.py	~96	15 min

Test checklist — run this after all Priority 1 tasks are done

- Send 5 messages — UI shows all 5 bubbles
- Send 35 messages — compression fires — UI still shows all 35 bubbles
- Check browser devtools → Network → SSE stream — control frame visible after token stream
- Check React state in devtools — contextMessages pruned, displayMessages intact
- Check POST body in devtools — rolling_summary is non-empty after first compression
- Send message 40 — response time should be similar to message 5
- GET /api/config — 200, no 500
- POST /api/verify-provider — 200 with success:true (not NameError crash)

Next document: Episodic Memory Layer — cross-session memory with LangMem.